

Bài 5: CÂY TỔNG QUÁT

Mục tiêu:

Về kiến thức:

+ Nắm vững các khái niệm cơ bản về mô hình Cây tổng quát, các phương pháp biểu diễn Cây tổng quát.

Về kỹ năng:

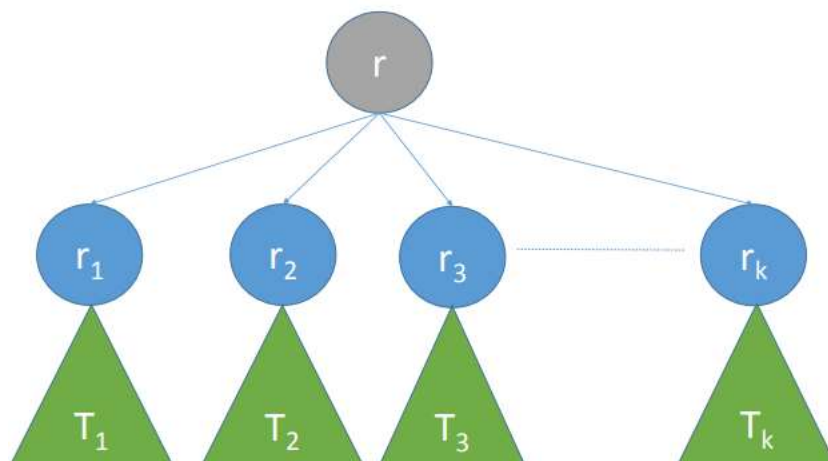
+ Vận dụng kiến thức đã học để cài đặt các phép toán cơ bản trên cây tổng quát trong các bài toán cụ thể.

5.1 Cây tổng quát

5.1.1 Định nghĩa cây và các khái niệm cơ bản trên cây

Có nhiều cách định nghĩa cây, sau đây ta định nghĩa bằng đệ quy như sau :

- 1) Tập có một đỉnh là một cây, cây này có gốc và lá là đỉnh duy nhất của nó.
- 2) Giả sử $T_1, T_2, \dots, T_k$ ($k \geq 1$) là các cây có gốc tương ứng là $r_1, r_2, \dots, r_k$; các cây $T_1, T_2, \dots, T_k$ đôi một không cắt nhau. Nếu r là một đỉnh không thuộc các cây $T_1, T_2, \dots, T_k$ và có quan hệ phân cấp (quan hệ cha - con) với các đỉnh $r_1, r_2, \dots, r_k$ thì tập T gồm đỉnh r và tất cả các đỉnh của các cây T_i làm thành một cây mới gốc r . Ta quy ước viết các đỉnh ở mức trên là các đỉnh cha, mức dưới là con



Hình 5.1: Cây tổng quát

Ví dụ về cây: Cách tổ chức bộ nhớ ngoài trong máy tính được tổ chức theo mô hình cây (gọi là cây thư mục)

Trong mô hình cây chứa tập hợp các đỉnh, giữa các đỉnh có một quan hệ phân cấp gọi là quan hệ cha - con. Ví dụ như hình 5.1: r gọi là cha của $r_1, r_2, \dots, r_k$

- Nếu tập các đỉnh = rỗng $\Rightarrow$ Cây là cây rỗng
- Số các cây con ở mỗi đỉnh được gọi là bậc của đỉnh đó. Đỉnh có bậc bằng 0 gọi là lá (đỉnh tận cùng), đỉnh không là lá gọi là đỉnh trong.
- Các đỉnh có cùng cha gọi là anh em, xét từ trái sang phải
- Tập các cây con phân biệt người ta gọi là Rừng
- Gốc của cây có mức 0 (level = 0). Nếu đỉnh cha có mức i thì các đỉnh con của nó sẽ có mức là $i+1$
- Chiều cao của cây (height) của cây là số mức lớn nhất của đỉnh có trên cây
- Một dãy các đỉnh $a_1, a_2, \dots, a_n$ ($n > 0$) sao cho a_i là cha của a_{i+1} gọi là một đường đi từ a_1 đến a_n với độ dài $n-1$. Luôn tồn tại đường đi từ gốc đến một đỉnh bất kỳ trong cây.
- Cây được sắp: Là cây mà các đỉnh trong cây được sắp xếp theo thứ tự nào đó. Nếu đỉnh a có các đỉnh con $b_1, b_2, \dots, b_m$ theo thứ tự này thì ta nói b_1 là con

trưởng và b_2 là em liền kề của nó.

5.1.2 Các phép toán cơ bản trên cây

Xét cây gốc T :

1) Hàm $parent(k, T)$ trả về nút cha của nút k trên cây T , nếu k là nút gốc thì hàm cho giá trị rỗng. Trong cài đặt cụ thể thì rỗng là một giá trị đặc biệt nào đó do ta chọn, nó phụ thuộc vào cấu trúc dữ liệu mà ta dùng để cài đặt cây.

2) Hàm $eldestChild(k, T)$ tìm con cả của nút k trên cây T , nếu k là lá thì hàm cho giá trị rỗng, ngược lại thì hàm sẽ trả về con trưởng của nút k

3) Hàm **$nextSibling(k)$** tìm em liền kề của nút k trên cây T , nếu k không có em liền kề thì hàm trả về giá trị rỗng

5.1.3 Các phép toán thăm (duyet) cây

* **Duyệt cây**: Là phép thăm các đỉnh trên cây, sao cho mỗi đỉnh chỉ được thăm duy nhất một lần.

* Xét ba phương pháp duyệt cây cơ bản:

- Duyệt cây theo thứ tự trước – *preOrder*
- Duyệt cây theo thứ tự giữa – *inOrder*
- Duyệt cây theo thứ tự sau - *postOrder*

Giả sử xét cây gốc T , các phương pháp duyệt tương ứng là:

1) Duyệt theo thứ tự trước (*preOrder*)

*** Nguyên tắc duyệt cây:**

- Nếu cây T rỗng: Ta không làm gì cả
- Nếu T khác rỗng: Trình tự thăm các đỉnh trên cây như sau:

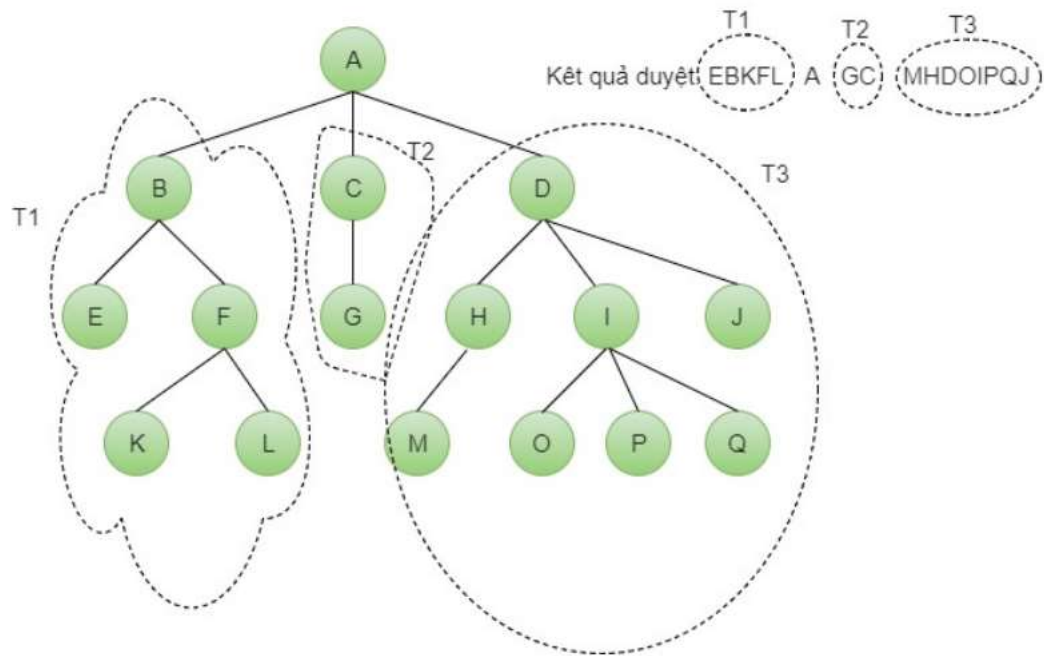
1) Thăm gốc T

2) Thăm các cây con: $T_1, T_2, \dots, T_k$ của T theo thứ tự trước

Ví dụ: Xét cây (xem hình 5.2)

- 1) Thăm cây con T_1 của T theo thứ tự giữa
- 2) Thăm gốc T
- 3) Thăm các cây con còn lại của T theo thứ tự giữa: $T_2, \dots, T_k$

Ví dụ: Xét cây (xem hình 5.3)



Hình 5.3: Thứ tự duyệt cây theo thứ tự giữa

***Chương trình có dạng như sau:**

```

void InOrder (Tree T)
{
    nodeType C;
    if (T==rong) printf ("Cay rong");
    else
    {
        C = eldestChild (T); // tim con truong
        if (C!= rong)
        {
            InOrder(C);
            C = nextSibling(C); //tim em lien ke
        }
        Visit(T); //tham goc
        while (C!=rong)
        {
            InOrder (C);
            C = nextSibling(C); //em lien ke
        }
    }
}

```

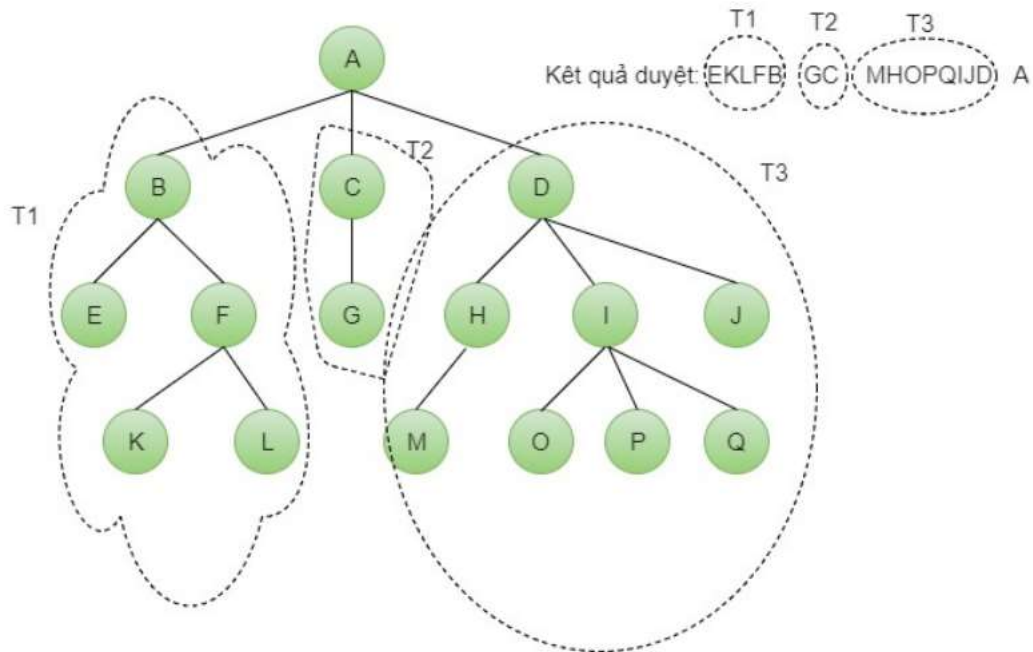
3) Duyệt theo thứ tự sau

*Nguyên tắc duyệt cây

- Nếu cây T rỗng: Ta không làm gì cả
- Nếu T khác rỗng: Trình tự thăm các đỉnh trên cây như sau:

- 1) Thăm cây con $T_1, T_2, \dots, T_k$ của T theo thứ tự sau
- 2) Thăm gốc T

Ví dụ: Xét cây (xem hình 5.4)



Hình 5.4: Thứ tự duyệt cây theo thứ tự sau

***Chương trình có dạng như sau:**

```
void PostOrder (Tree T)
{
    nodeType C;
    if (T==rong) printf("cay rong");
    else
    {
        C = eldestChild(T); // con truong
        while (C!= rong)
        {
            PostOrder(C);
            C = nextSibling(C); // em lien ke
        }
        Visit(T); //Tham cha hay in cha
    }
}
```

5.1.4 Biểu diễn cây trên máy tính

Chúng ta xét 3 phương pháp biểu diễn cây cơ bản:

- + Biểu diễn cây bằng danh sách các con của mỗi đỉnh
- + Biểu diễn cây bởi con trưởng và em liền kề của mỗi đỉnh
- + Biểu diễn cây bởi cha của mỗi đỉnh

Với mỗi phương pháp ta đều có thể sử dụng mảng hoặc con trỏ hoặc kết hợp giữa mảng và con trỏ để biểu diễn cây trên máy tính. Với mỗi dạng biểu diễn ta thu được một cấu trúc dữ liệu cây tương ứng.

5.1.4.1 Biểu diễn cây bằng danh sách các con của mỗi đỉnh

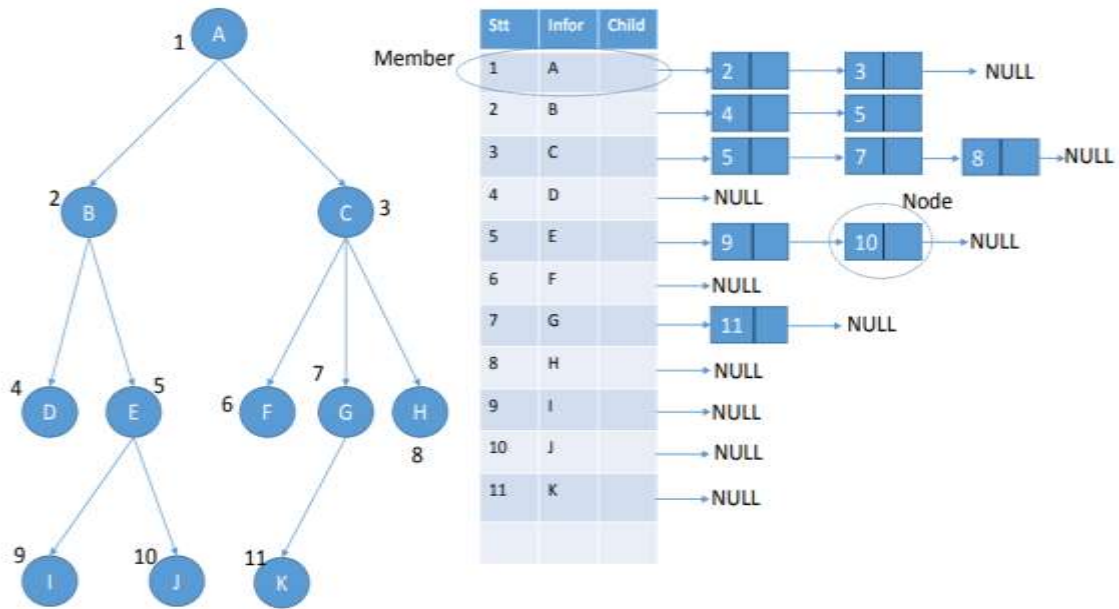
Mỗi đỉnh trong cây đi kèm với một danh sách các con của nó. *Tổ chức lưu trữ* các phần tử trong danh sách và lưu trữ các đỉnh của cây như thế nào là phụ thuộc vào các đặc trưng và yêu cầu của bài toán cụ thể.

a) Sử dụng mảng để biểu diễn cây

Các đỉnh trên cây được tổ chức lưu trữ trong một mảng kế tiếp. Danh sách các con của mỗi đỉnh trên cây có thể được lưu trữ dưới dạng danh sách kế tiếp, danh sách liên kết đơn cũng có thể là danh sách liên kết kép hoặc vòng.....

Giả sử ta tổ chức lưu trữ danh sách các con của mỗi đỉnh trên cây trong một danh sách liên kết đơn => ta có dạng biểu diễn Tree1, còn gọi là cấu trúc dữ liệu Tree1

Ví dụ: Xét cây, các đỉnh trên cây được đánh số thứ tự từ trên xuống, từ trái qua phải và hình ảnh cây sau khi biểu diễn với dạng Tree1 (hình 5.5)



Hình 5.5: Biểu diễn cây bằng danh sách con của mỗi đỉnh

Trong hình 5.5 mỗi con trong d/s các con gồm 2 trường: ID và next, mỗi đỉnh của cây gồm 2 trường : Infor và Childs, các đỉnh của cây được lưu trong mảng kế tiếp n phần tử

* **Dạng cài đặt tương ứng sẽ là:**

```
typedef struct Member
{
    int Infor;
    struct node* Child;
}Member;
```

```
typedef struct node
{
    int ID;
    node *Next;
}node;
```

\\cay lúc này ban chất là mảng các Member

```
typedef struct Tree
{
    Member elems[N];
    int size;
}Tree;
```

Nhận xét:

Cây sau khi biểu diễn trên máy tính được xem như một mảng các danh sách liên kết đơn. Thực hiện các thao tác trên cây là thực hiện các thao tác trên mảng các danh sách này

*** Các phép toán tác động trên cây**

1 - Cho một đỉnh thứ k trên cây T, tìm con trưởng của nó

Với cách cài đặt này phép toán tìm con trưởng được xác định rất dễ dàng thông qua thành phần Child của nút thứ k trên cây

Chương trình cụ thể như sau:

```
int eldestChild(int k, Tree T)
{
    if (T.elems[k].Child != NULL)
        return( T.elems[k].Child->ID);
    else return -1;
}
```

2 - Cho một đỉnh thứ k trên cây T, Tìm cha của nó

Phép toán này được mô tả như sau: Duyệt các nút trên cây. Tại mỗi nút duyệt các con của nó. Nếu con của nó có chứa nút k thì kết luận nút đang xét là cha của k. Ngược lại nếu không có con nào thì chuyển sang nút tiếp.

Ví dụ: Hình 5.5 Tìm cha của nút thứ 7 trên cây. Với cách cài đặt như trên ta thực hiện như sau: Tại nút thứ 1, duyệt các con của nó thấy các con của nó là 2 và 3 khác k (k=7) => chuyển sang nút số 2. Tại nút số 2 có các con là 4 và 5 (khác k) => chuyển sang nút số 3. Tại nút k có con là 5, 7 và 8. Dừng lại và kết luận cha là 3

Phép toán như sau:

```

int Parent(int k, Tree T)
{
    int i;
    struct node* p;
    i = 0;
    while (i < T.size)
    {
        p = T.elems[i].Child;
        while (p != NULL)
            if (p->ID == k)
                return(i);
            else p = p->Next;
        i++;
    }
    return(-1);
}

```

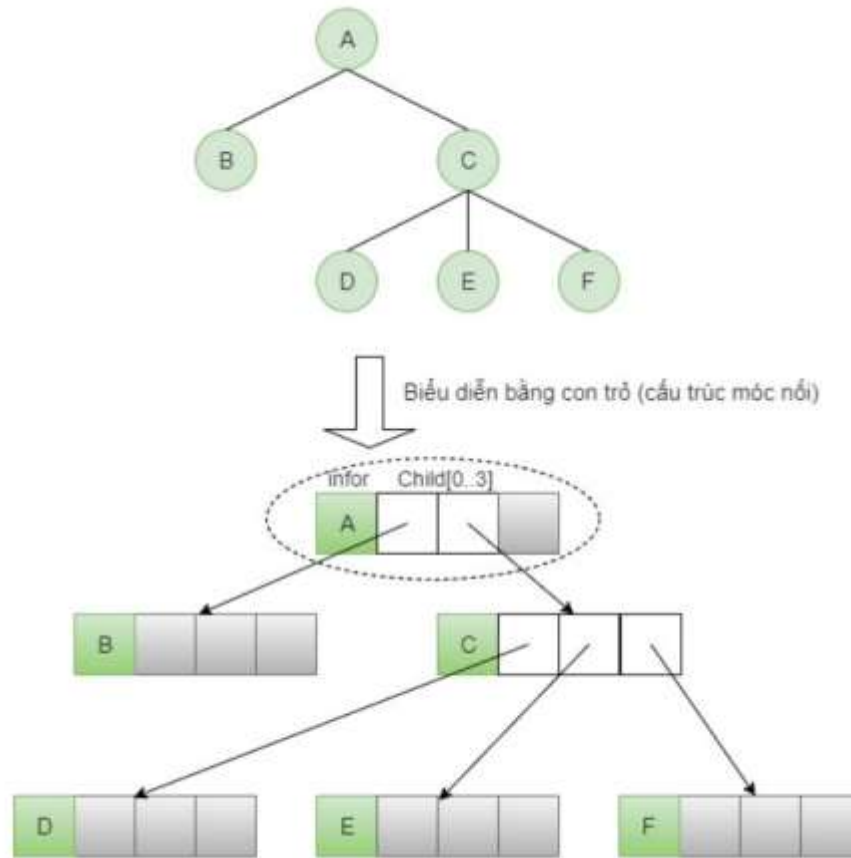
b) Sử dụng con trỏ để biểu diễn cây

* Mô tả dạng biểu diễn

- Các đỉnh của cây được lưu trữ ở những ô nhớ không kế tiếp nhau, liên kết với nhau thông qua cơ chế lưu địa chỉ của nhau

- Danh sách các con của mỗi đỉnh có thể được lưu trữ trong danh sách kế tiếp hoặc danh sách móc nối. Nếu ta biết trước số con tối đa của mỗi đỉnh trên cây, ta có thể dùng mảng để lưu danh sách các con này. Dạng biểu diễn này giả sử ta gọi là Tree2

Ví dụ: Xét cây và hình ảnh cây sau khi biểu diễn dưới dạng Tree2 được mô tả trong hình 5.6:



Hình 5.6: Biểu diễn con bằng móc nối các nút (con trỏ)

Trong hình 5.6 giả sử số con tối đa của mỗi đỉnh là k (có thể xem là cây k phân), mỗi đỉnh/nút là một cấu trúc gồm 2 trường:

- + Trường *Infor*: Lưu dữ liệu của đỉnh
- + Trường *Child*: Là một mảng chứa tối đa k con trỏ, mỗi con trỏ trỏ tới một gốc cây con

* Dạng cài đặt

```
#define K 100; //{K là bậc của cây}
typedef struct Node
{
    int Info;
    struct Node *childs[K] ; //mang các con
} Node;
typedef struct Node* Tree;
```

Nhận xét:

- Để quản lý cây người ta dùng 1 con trỏ luôn trỏ tới gốc cây, giả sử con trỏ ROOT
- Cây rỗng khi ROOT = NULL (trỏ tới đất/không trỏ tới đâu cả). Để truy cập đến các đỉnh trên cây chúng ta truy cập tuần tự, xuất phát từ gốc.

*** Cài đặt các phép toán cơ bản với dạng cài đặt này:**

Xem như bài tập dành cho bạn đọc

5.1.4.2 Biểu diễn cây bằng con trỏ và em liên kế của mỗi đỉnh

Mô tả phương pháp biểu diễn:

- Mỗi đỉnh trên cây cần lưu trữ 3 thông tin:
 - Dữ liệu của đỉnh (infor)
 - Con trưởng của đỉnh (EldestChild): Lưu ID
 - Em liên kế của đỉnh (NextSibling): Lưu ID

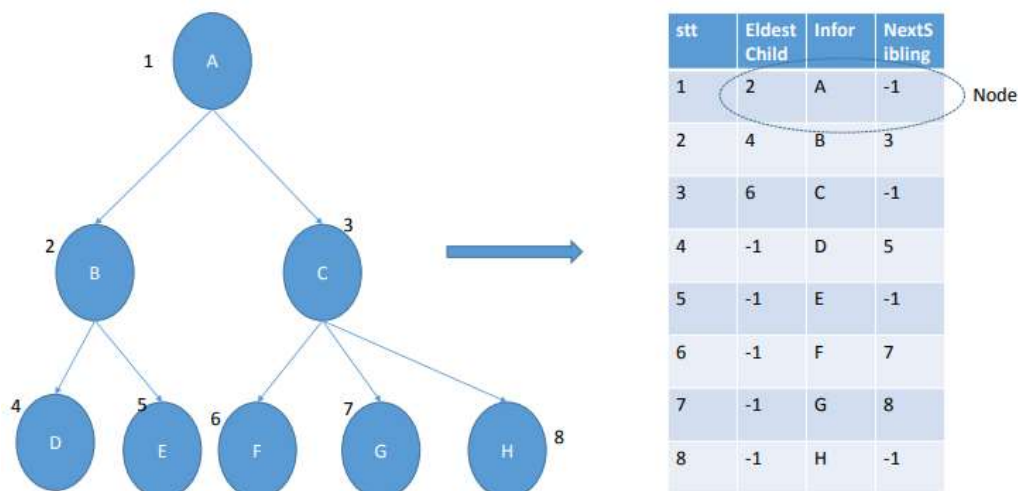
=> Mỗi đỉnh, còn gọi là một nút có thể xem như một cấu trúc bản ghi gồm 3 trường: *infor, EldestChild, NextSibling*

- Các đỉnh trên cây được tổ chức lưu trữ kế tiếp trong 1 mảng gồm tối đa n phần tử hoặc lưu trữ dưới dạng móc nối. Ta lần lượt xét 2 dạng biểu diễn này

Biểu diễn cây bởi mảng :

- Các đỉnh trên cây được tổ chức lưu trữ kế tiếp trong 1 mảng gồm tối đa n phần tử. Mỗi đỉnh được tổ chức lưu trữ trong 1 cấu trúc gọi là nút được mô tả ở trên

Ví dụ: Xét cây và hình ảnh cây sau khi biểu diễn (hình 5.7):



Hình 5.7: Biểu diễn cây qua con trưởng và em liên kế

Dạng cài đặt

```
typedef struct Node
{
    Item Info; // Infor chứa dữ liệu có kiểu Item
    int EldestChild ;// ID của con trưởng
    int NextSibling ; //ID của em liên kế
}Node;
typedef struct Tree
{
    struct Node Elems[N];
    int size;
}Tree;
```

Nhận xét

- Cây sau khi biểu diễn được xem như là một mảng các nút
- Thao tác trên cây là thao tác trên mảng các nút này

b) Cài đặt các phép toán cơ bản trên cây

Với cách cài đặt này ta thấy dễ dàng tìm con trưởng và em liên kế của đỉnh thứ k trên cây T – xem như bài tập dành cho bạn đọc, ở đây ta xét phép toán tìm cha của đỉnh k trên cây T. Để tìm cha của đỉnh thứ k trên cây T, ta làm như sau:

Duyệt lần lượt các đỉnh trên cây, xuất phát từ gốc (i = 1), kiểm tra xem con trưởng j của i có = k hay không? Nếu có bằng nhau thì i chính là cha cần tìm, ngược lại, kiểm tra

em liền kề của con trưởng j xem có $= k$ hay không, nếu bằng nhau thì i cũng chính là cha cần tìm, nếu không kiểm tra j là em liền kề của j cũ, cứ tiếp tục như vậy cho đến khi kiểm tra hết các con của i mà không có con nào $= k$ thì duyệt đỉnh i tiếp theo trên cây, ngược lại chỉ cần 1 con j của i mà $= k$ thì ta dừng việc tìm kiếm và kết luận i chính là đỉnh cha cần tìm.

Viết chương trình coi như là bài tập.

Biểu diễn cây bởi con trỏ

Mô tả dạng biểu diễn:

- Mỗi đỉnh trên cây cần lưu trữ 3 thông tin:

Dữ liệu của đỉnh (infor)

Con trưởng của đỉnh (EldestChild): trỏ tới gốc cây con trưởng

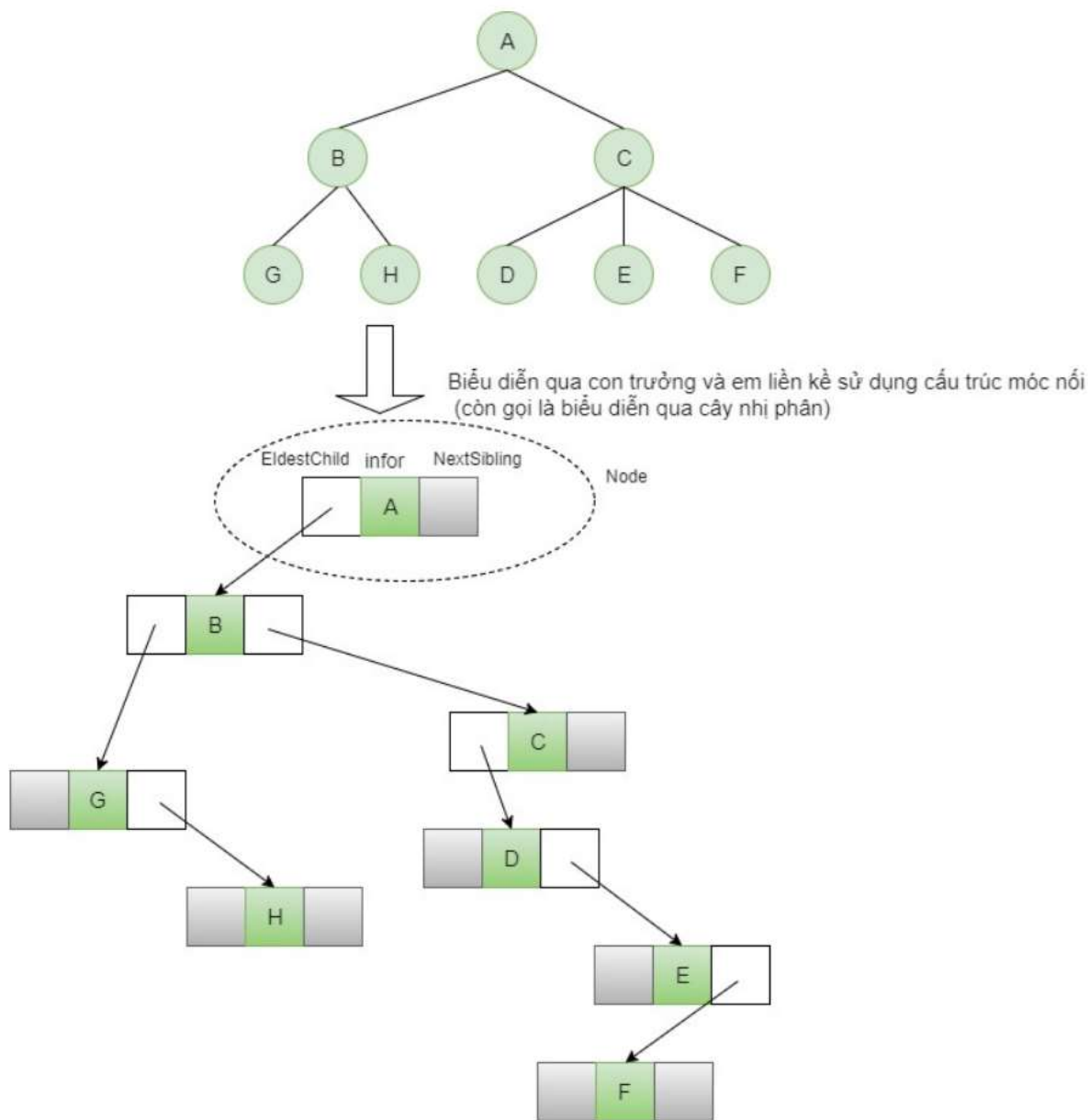
Em liền kề của đỉnh (NextSibling): Trỏ tới gốc cây em kề

=> *Mỗi đỉnh có thể xem như một cấu trúc bản ghi gồm 3 trường: infor, EldestChild, NextSibling.* Cấu trúc 1 nút có dạng như sau:

eldestchild	Info	Nextsibling
-------------	------	-------------

- Các đỉnh trên cây được tổ chức lưu trữ móc nối thông qua cơ chế lưu địa chỉ

Ví dụ: Xét cây và hình ảnh cây sau khi biểu diễn (hình 5.8)



Hình 5.8: Biểu diễn cây qua con trỏ và em liên kế sử dụng con trỏ

Với cách cài đặt này, để quản lý cây, người ta dùng một con trỏ luôn trỏ tới gốc cây, giả sử con trỏ Root. Khi đó cây rỗng khi $Root = NULL$. Để truy cập đến các đỉnh trên cây ta truy cập tuần tự xuất phát từ gốc. Cách biểu diễn này là cách chuyển cây đa phân trở về cây nhị phân tương ứng.

5.1.4.3 Biểu diễn cây bởi cha của mỗi đỉnh

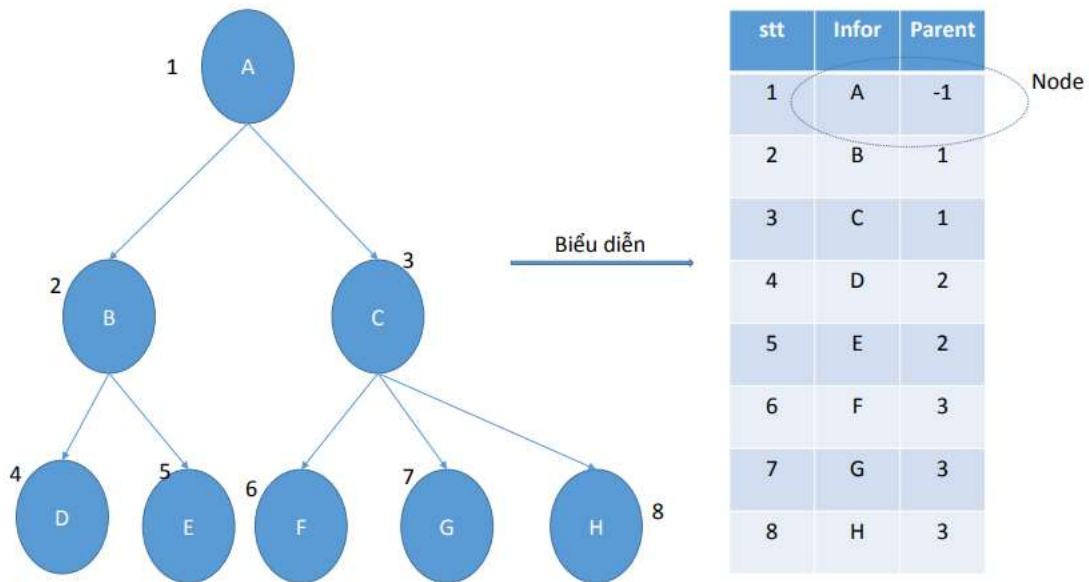
Cách biểu diễn:

- Đánh số các đỉnh trên cây (có thể bắt đầu từ 0 hoặc 1 tùy ngôn ngữ lập trình)
- Với mỗi đỉnh lưu 2 trường thông tin: *infor* và *parent*. Trong đó *infor* là dữ liệu của đỉnh, *parent* là cha của đỉnh đó (thể hiện qua *ID* của cha)
- Cây lúc này là tập các đỉnh như trên.

Cài đặt như sau:

```
#define N 100 ;//phan tu toi da cua cay
typedef struct Node
{
    Item info;
    int parent;
}Node;
typedef struct Tree
{
    struct Node elems[N];
    int size;
}Tree;
```

Ví dụ: Xét cây



Hình 5.8: Biểu diễn cây qua nút cha

5.3 Thực hành

5.3.1 Mục tiêu

Bài thực hành giúp sinh viên biết cách cài đặt cây đa phân. Sau đó, sinh viên thực hành thành thạo trên các phép toán của cây đa phân. Cuối cùng, sinh viên có khả năng vận dụng vào triển khai bài toán ứng dụng

5.3.2 Bài thực hành mẫu

Bài 1: Cài đặt cây nhị phân sử dụng mảng có khóa là các số nguyên. Viết chương trình nhập và in cây đa phân theo thứ tự trước, giữa, sau.

```
#include <stdio.h>
#include <conio.h>
#define max 100
#define MAXLENGTH 100      //chi so toi da cua mang
#define NIL -1
typedef int DataType;
typedef int Node;
typedef struct {
    DataType Data[MAXLENGTH]; //Luu gia tri cua nut
    int MaxNode;
}Tree;
//Kiem tra cay rong
int EmptyTree(Tree T) {
    return T.MaxNode == 0;
}
//Xac dinh nut goc trong cay
Node Root(Tree T) {
    if (!EmptyTree(T))
        return 0;
    else
        return NIL;
}
//con trai cua nut p
int Left_Child(Node p, Tree T) {
    return 2*(p+1) - 1;
}
//con phai cua nut p
int Right_Child(Node p, Tree T) {
    return 2*(p+1);
}
//duyet tien tu
```

```

void NLR(Node p, Tree T) {
    if(T.Data[p]!=NIL) {
        printf("%d ", T.Data[p]);
        NLR(Left_Child(p,T),T);
        NLR(Right_Child(p,T),T);
    }
}

//duyet trung tu
void LNR(Node p, Tree T) {
    if(T.Data[p]!=NIL) {
        LNR(Left_Child(p,T),T);
        printf("%d ", T.Data[p]);
        LNR(Right_Child(p,T),T);
    }
}

//duyet hau tu
void LRN(Node p, Tree T) {
    if(T.Data[p]!=NIL) {
        LRN(Left_Child(p,T),T);
        LRN(Right_Child(p,T),T);
        printf("%d ", T.Data[p]);
    }
}

/*doc cay
neu khong co con trai hoac phai thi nhap -1*/
void Read_Tree(Tree * T) {
    int i = 0, Child = 0;
    printf("Nhap vao so nut: ");
    scanf("%d",&(*T).MaxNode);
    while(i<(*T).MaxNode) {
        if(i==0)
        {
            printf("Nut goc:");
            scanf("%d",&(*T).Data[i]);
            (*T).Data[Left_Child(i,*T)] = NIL;
            (*T).Data[Right_Child(i,*T)] = NIL;
            i++;
        }
        else
            if((*T).Data[Child]!=NIL)
            {
                int k;
                printf("Con trai %d: ",Child);
                k = Left_Child(Child,*T);
            }
    }
}

```

```

scanf("%d",&(*T).Data[k]);
if((*T).Data[k] != NIL) {
    (*T).Data[Left_Child(k,*T)] = NIL;
    (*T).Data[Right_Child(k,*T)] = NIL;
    i++;
}
printf("Con phai %d: ",Child);
k = Right_Child(Child,*T);
scanf("%d",&(*T).Data[k]);
if((*T).Data[k] != NIL) {
    (*T).Data[Left_Child(k,*T)] = NIL;
    (*T).Data[Right_Child(k,*T)] = NIL;
    i++;
}
Child++;
}
}
}
int main() {
    Tree T;
    //clrscr();
    Read_Tree(&T);
    printf("Duyet tien tu.\n");
    NLR(Root(T),T);
    printf("\nDuyet trung tu.\n");
    LNR(Root(T),T);
    printf("\nDuyet hau tu.\n");
    LRN(Root(T),T);
    getch();
    return 1;
}

```

5.3.3 Bài tập luyện tập

Bài 3: Cài đặt cây đa phân sử dụng con trỏ có dữ liệu là xâu kí tự. Viết hàm tìm độ sâu (chiều cao) của cây. Viết chương trình nhập, in cây vừa nhập và cho biết cây vừa nhập có độ cao bằng bao nhiêu.

Bài 4: Cài đặt cây đa phân sử dụng con trỏ có dữ liệu chứa kiểu số nguyên. Viết hàm tính số lượng nút lá trên cây đa phân

Bài 5: Cài đặt cây đa phân sử dụng con trỏ có dữ liệu chứa kiểu số nguyên. Viết hàm tìm

giá trị chẵn lớn nhất trên cây đa phân

Bài 6: Cài đặt cây đa phân sử dụng con trỏ có dữ liệu chứa kiểu số nguyên. Viết hàm tính tổng các số trên cây đa phân.